

Lab 2: Investigating Process Lifecycles and OS Interaction

Introduction

This lab shows that a C program is not just code that prints text: once it runs, it becomes a **process** managed by the Linux kernel. Five small programs were written to query a process's identity, control how long it runs, read input, and report success or failure back to the shell.

Objectives

- Run a long-lived process in the background and monitor it from a second terminal.
- Read a process's own ID (PID) and its parent's ID (PPID) with system calls.
- Return exit codes to the OS and read them with `$?`.
- Use the standard input and output streams (stdin and stdout).
- Make a program branch on user input and end with a matching exit code.

Lab environment

Item	Used in this lab
Operating system	Ubuntu Linux running in a Docker container (<code>oslab</code>)
Compiler	gcc (from the <code>build-essential</code> package)
Editor	nano
Process tools	<code>ps</code> (from the <code>procps</code> package)
Working folder	<code>/labs/lab2</code> , mounted from the host computer so files persist

The container was started with `docker start -ai oslab` . A second terminal was opened with `docker exec -it oslab bash` , which gives a separate shell inside the same container so running processes can be inspected.

Task 1: The long-running process

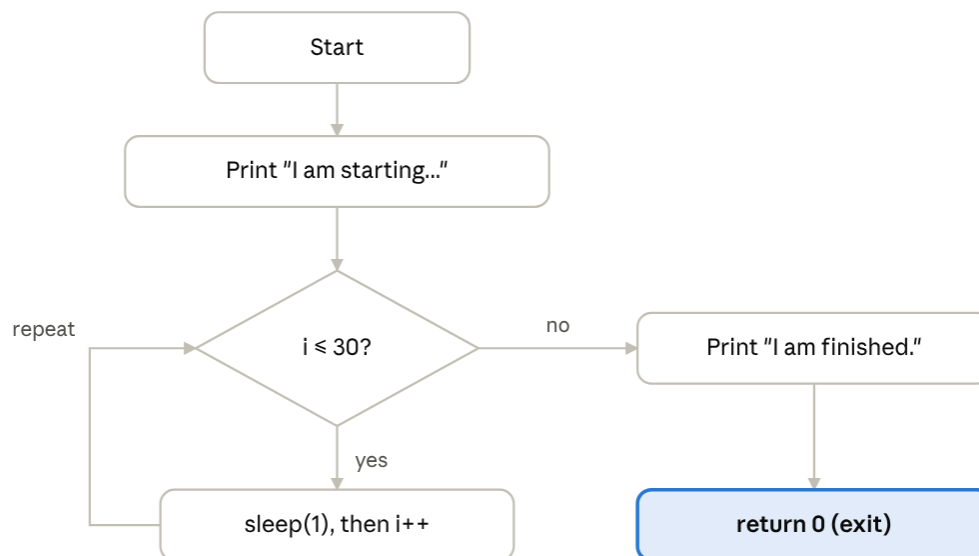
A process stays alive in the kernel's process table for as long as it runs, and other tools can inspect it during that time. This program lives for about 30 seconds so it can be monitored from a second terminal.

Theory

- **Process:** a program in execution. The kernel gives it a PID, memory and a state.
- **sleep(1)** : a system call that pauses the process for one second. While waiting, the kernel puts it in the **S (sleeping)** state and it uses almost no CPU.
- **Background job (&):** the shell starts the program but returns the prompt immediately, so the terminal can be used while the program keeps running.
- **ps aux** : lists every process on the system with its owner, PID, CPU and memory use, terminal (TTY) and state (STAT).

Flow diagram

The loop sleeps one second at a time until 30 seconds pass



Task 1 flow · a 30-step sleep loop

The loop repeats 30 times; each pass sleeps one second, so the process stays alive for about 30 seconds.

Source code (`alive.c`)

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");

    // Loop for 30 seconds
    for (int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }

    printf("I am finished.\n");
    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
printf("I am starting...\n")

// Loop for 30 seconds
for(int i = 1; i <= 30; i++) {
sleep(1); // Pauses execution for 1 second
}
printf("I am finished.\n");
return 0;
}
```

Execution

Terminal 1 compiled the program and ran it in the background:

```
gcc alive.c -o loop
./loop &
```

```

root@067c2baa3bec:/labs/lab2# ./loop &
[1] 48
root@067c2baa3bec:/labs/lab2# I am starting...
I am finished.

[1]+  Done                  ./loop
root@067c2baa3bec:/labs/lab2#

```

The shell printed `[1] 48` : job number 1 with PID 48. The prompt returned at once, and "I am finished." appeared 30 seconds later, followed by `[1]+ Done ./loop`.

Verification from a second terminal

A second shell was opened with `docker exec -it oslab bash`, and the running process was searched for:

```
ps aux | grep loop
```

```

Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\wwwan> docker exec -it oslab bash
root@067c2baa3bec:/# ps aux | grep loop
root      45  0.0  0.0   2684  1096 pts/0    S+   14:40   0:00 ./loop
root      47  0.0  0.0   3532  1612 pts/1    S+   14:40   0:00 grep --colo
r=auto loop
root@067c2baa3bec:/# |

```

Column	Value	Meaning
USER	root	Owner of the process
PID	45	Unique process ID
%CPU / %MEM	0.0 / 0.0	Almost no resources, because it is sleeping
RSS	1096 KB	Physical memory in use
TTY	pts/0	Belongs to terminal 1
STAT	S+	Sleeping (S); the + means foreground process group
COMMAND	./loop	The program

The second line is the `grep loop` command itself on `pts/1`, which proves the check ran from a different terminal.

Running the same command after 30 seconds shows only the `grep` line, because the process had finished and was removed from the process table:

```
root@067c2baa3bec:/# ps aux | grep loop
root      55  0.0  0.0  3532 1620 pts/1    S+   14:43   0:00 grep --colo
r=auto loop
```

Observations

- A process is visible to the whole system while it is alive and disappears once it exits.
- `sleep()` keeps the process in state S with 0.0% CPU, so a waiting process does not waste the processor.
- `grep` always matches itself; `ps aux | grep [1]oop` hides that extra line.

Task 2: Process identity (PID and PPID)

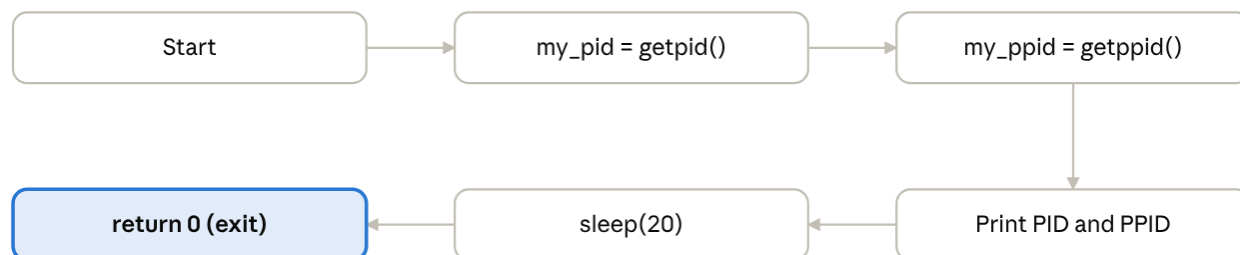
The program read its own PID (65) and its parent's PID (1) directly from the kernel. The parent is the shell that launched it, which in this Docker container is PID 1.

Theory

- **PID (Process ID)**: a unique number the kernel assigns to every process. No two running processes share one.
- **PPID (Parent Process ID)**: the PID of the process that created this one. Every process except the first has a parent, forming a process tree.
- `getpid()` / `getppid()`: system calls from `<unistd.h>` that return the PID and PPID.
- `pid_t`: the data type used to store process IDs.

Flow diagram

The program asks the kernel for its own PID and its parent PID



Task 2 flow · two system calls, then a 20-second sleep

The 20-second sleep keeps the process alive long enough to check its PID with `ps`.

Source code (`PID.c`)

```
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

```
GNU nano 7.2 PID.c
#include <stdio.h>
#include <unistd.h>

int main() {
// Get current Process ID
pid_t my_pid = getpid();

// Get Parent Process ID
pid_t my_ppid = getppid();

printf("My PID is: %d\n", my_pid);
printf("My Parent PID is: %d\n", my_ppid);
printf("Sleeping for 20 seconds...\n");
sleep(20);

return 0;
}
```

Execution

```
gcc PID.c -o pid
./pid
```

```
root@067c2baa3bec:/labs/lab2# gcc PID.c -o pid
root@067c2baa3bec:/labs/lab2# ./pid
My PID is: 65
My Parent PID is: 1
Sleeping for 20 seconds...
root@067c2baa3bec:/labs/lab2# |
```

Why the parent PID is 1

On a normal Linux machine, PID 1 is `init` or `systemd` and the terminal shell has a larger PID. A Docker container has its own process namespace, and the first process started in it becomes PID 1. Here that first process is the `bash` shell opened by `docker start -ai oslab`, so the shell that ran `./pid` is PID 1. The program's parent is therefore still the terminal shell, exactly as the theory says.

This can be confirmed while the program sleeps:

```
./pid &
ps -p <PID> -o pid,ppid,cmd    # shows the same PID and PPID
echo $$                       # prints the shell's own PID: 1
```

Observations

- The PID is assigned by the kernel and changes every time the program runs.
- The PPID links the program to the shell that started it.
- Containers isolate processes: PIDs inside the container start again from 1.

Task 3: Exit codes and OS feedback

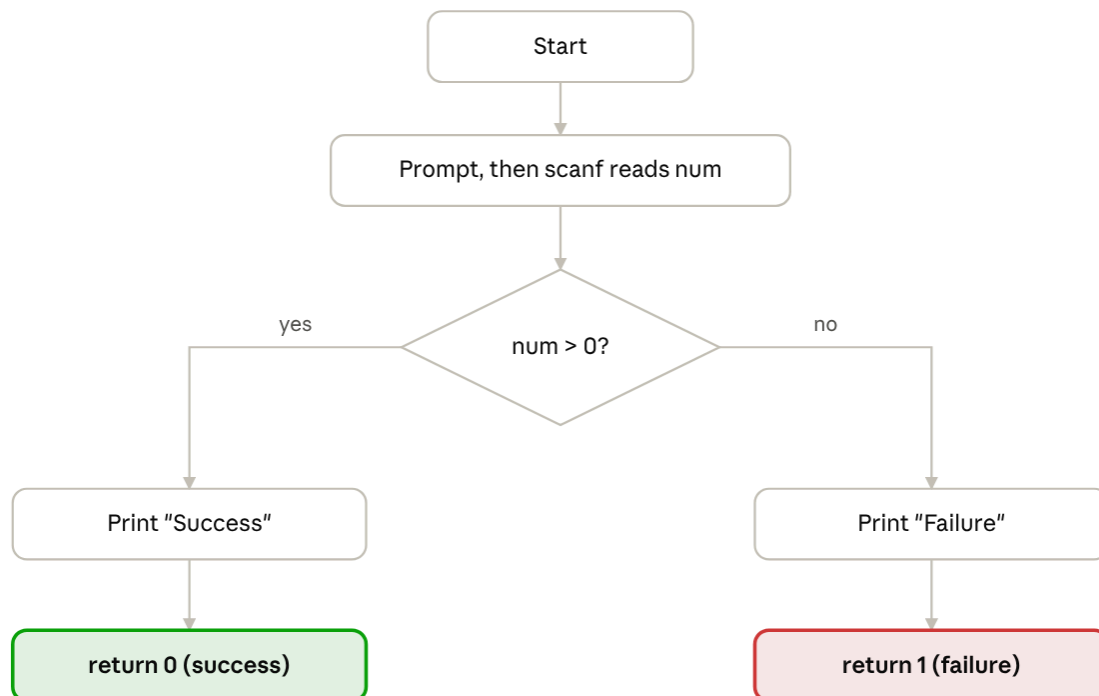
The program returned 0 for a positive number and 1 for a negative one, and the shell reported both values through `$?`.

Theory

- **Exit code (exit status):** the integer a process hands back to the OS when it ends. The value of `return` in `main()` becomes the exit code.
- **0 means success;** any non-zero value (such as 1) means an error or failure.
- **`$?`**: a shell variable holding the exit code of the last command. It must be read immediately, because the next command overwrites it.
- Scripts and automation pipelines use exit codes to decide whether to continue or stop.

Flow diagram

The sign of the number decides which exit code the OS receives



Task 3 flow · one decision, two exit codes

Only a number greater than zero reaches `return 0`; zero or a negative number ends with `return 1`.

Source code (`exit.c`)

```
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

```
GNU nano 7.2 exit.c
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success,
negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

Execution and verification

```
gcc exit.c -o exit
./exit      # enter 78
echo $?
./exit      # enter -78
echo $?
```

```
root@067c2baa3bec:/labs/lab2# gcc exit.c -o exit
root@067c2baa3bec:/labs/lab2# ./exit
Enter a number (positive for success,negative for fail): 78
Success
root@067c2baa3bec:/labs/lab2# echo $?
0
root@067c2baa3bec:/labs/lab2# ./exit
Enter a number (positive for success,negative for fail): -78
Failure
root@067c2baa3bec:/labs/lab2# echo $?
1
root@067c2baa3bec:/labs/lab2# |
```

Input	Program output	echo \$?	Meaning
78	Success	0	Success reported to the OS
-78	Failure	1	Failure reported to the OS

Observations

- The number returned from `main()` is exactly what the shell stores in `$?`.
- The program's printed message and its exit code are separate: the message is for the user, the exit code is for the OS.
- An input of 0 also counts as failure, because the condition is `num > 0`.
- While copying the code from the lab sheet, the long `printf` line was split across two lines, which caused a "missing terminating " character" compile error. A string in C must stay on one line.

Task 4: Standard I/O streams

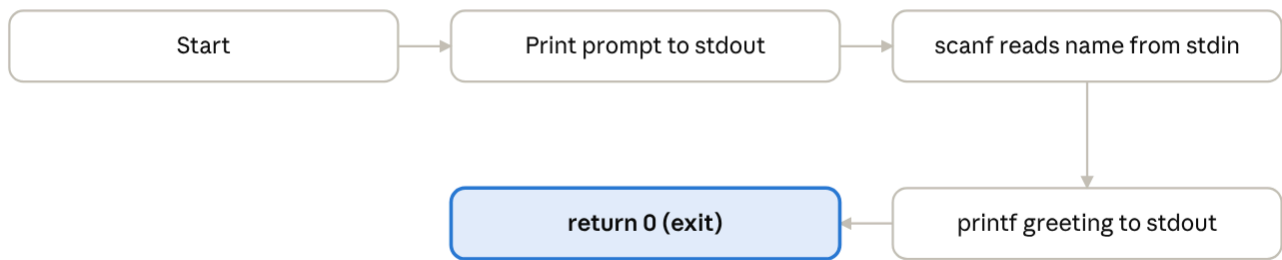
The program read a name from standard input and greeted the user on standard output. Entering "Anish Karki" printed only "Anish", because `scanf("%s")` stops reading at the first space.

Theory

- Every process starts with three streams opened by the OS:
 - **stdin** (file descriptor 0): input, normally the keyboard
 - **stdout** (file descriptor 1): normal output, normally the terminal
 - **stderr** (file descriptor 2): error output
- `scanf()` reads from stdin and `printf()` writes to stdout.
- Because they are streams, the shell can redirect them, for example `echo Anish | ./input` feeds the name through a pipe instead of the keyboard.

Flow diagram

Input arrives on stdin and the greeting leaves on stdout



Task 4 flow · stdin in, stdout out

The program waits at `scanf` until a line is typed into stdin.

Source code (`input.c`)

```
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```

```
#include <stdio.h>

int main() {
char name[50];

printf("Enter your name: ");
// Read string from standard input
scanf("%s", name);

printf("Hello, %s! Welcome to OS Class.\n",name);
return 0;
}
```

Execution

```
gcc input.c -o input
./input
```

```
root@067c2baa3bec:/labs/lab2# gcc input.c -o input
root@067c2baa3bec:/labs/lab2# ./input
Enter your name: Anish Karki
Hello, Anish! Welcome to OS Class.
root@067c2baa3bec:/labs/lab2# |
```

Observations

- `scanf("%s")` treats a space as the end of the word, so the surname "Karki" stayed unread in `stdin`. `fgets(name, sizeof(name), stdin)` would read the whole line.
- **Security note:** `%s` has no length limit. Typing more than 49 characters would write past the end of the 50-byte `name` array, a **buffer overflow**, which attackers can use to crash or take over a program. A safer format is `scanf("%49s", name)`.

Task 5: Conditional execution and termination

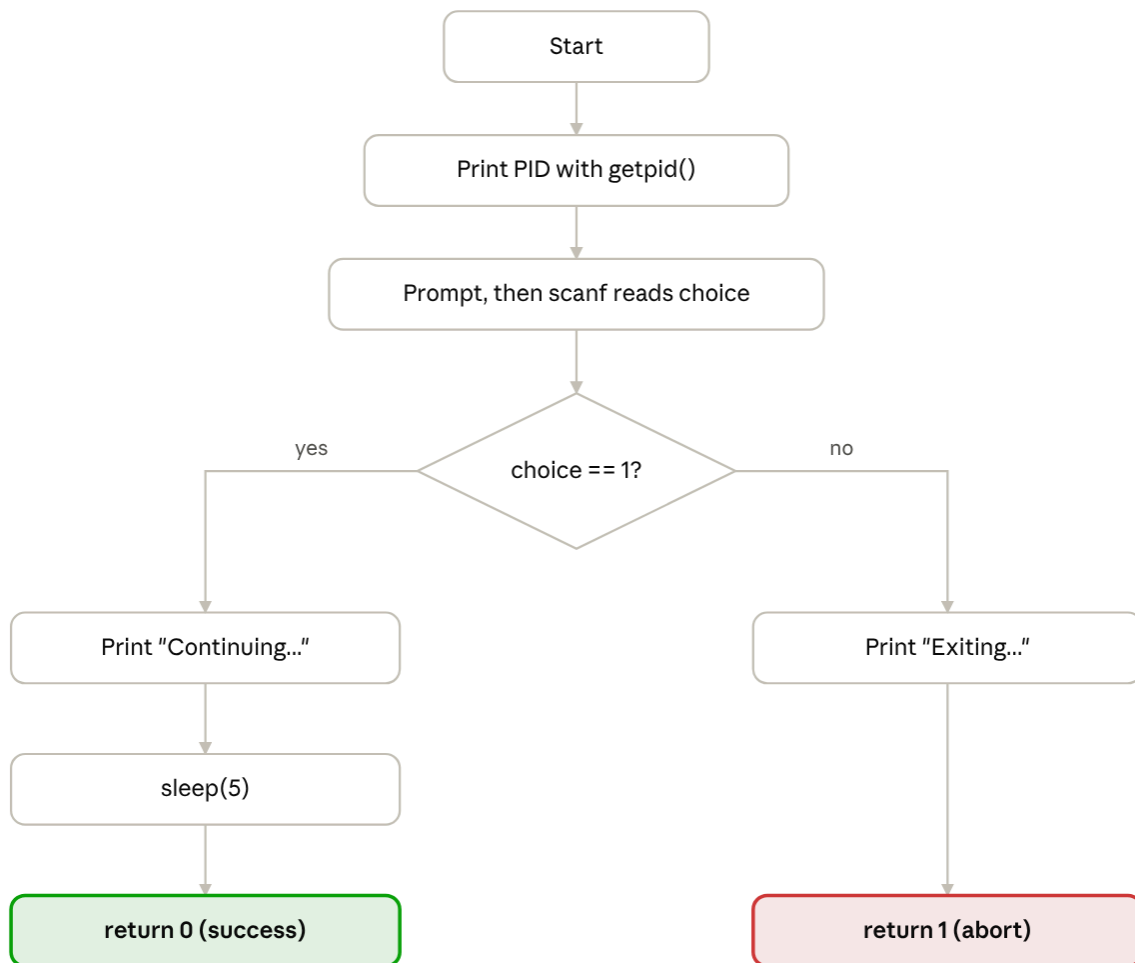
The program branched on the user's choice: "1" continued and exited with 0, "0" aborted and exited with 1. Each run received a new PID (135, then 136).

Theory

- **Conditional execution:** an `if / else` lets the program take different paths depending on input.
- **Termination:** each path ends with its own `return`, so the OS learns which path was taken through the exit code.
- **Process creation:** every time a command is run, the shell creates a brand-new process, so the PID changes between runs.
- Shells and pipelines use these exit codes, for example `./conditional && echo next` runs the second command only on success.

Flow diagram

The choice decides whether the program continues or aborts



Task 5 flow · continue or abort

Any answer other than 1 takes the abort path and returns 1.

Source code (`conditional.c`)

```

#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    // Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);
  
```

```

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}

```

```

#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    // Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}

```

Execution and verification

```

gcc conditional.c -o conditional
./conditional # enter 1
echo $?
./conditional # enter 0
echo $?

```

```

root@067c2baa3bec:/labs/lab2# ./conditional
Current PID: 135
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
root@067c2baa3bec:/labs/lab2# echo $?
0
root@067c2baa3bec:/labs/lab2# ./conditional
Current PID: 136
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
root@067c2baa3bec:/labs/lab2# echo $?
1
root@067c2baa3bec:/labs/lab2# |

```

Run	PID	Choice	Output	echo \$?
1	135	1	Continuing... (then a 5-second pause)	0
2	136	0	Exiting...	1

Observations

- The PID went from 135 to 136, showing that each run is a separate process.
- The "Yes" path paused for 5 seconds before ending, because of `sleep(5)`.
- The exit code tells a calling script whether the user chose to continue or abort.

Conclusion

The five programs showed that a C program becomes a process managed by the Linux kernel. Using `getpid()`, `getppid()`, `sleep()` and return codes, each program interacted with the OS: it had an identity (PID and PPID), a lifetime that could be monitored with `ps`, input and output streams, and an exit code the shell could read through `$?`. Running everything inside Docker also showed that containers isolate processes, since the container's shell was PID 1.

GitHub repository

The source files and this documentation are pushed to a personal GitHub repository with a `README.md`.

File	Task
<code>alive.c</code>	Long-running process
<code>PID.c</code>	PID and PPID
<code>exit.c</code>	Exit codes
<code>input.c</code>	Standard I/O streams
<code>conditional.c</code>	Conditional execution

Repository link: *add your GitHub URL here*